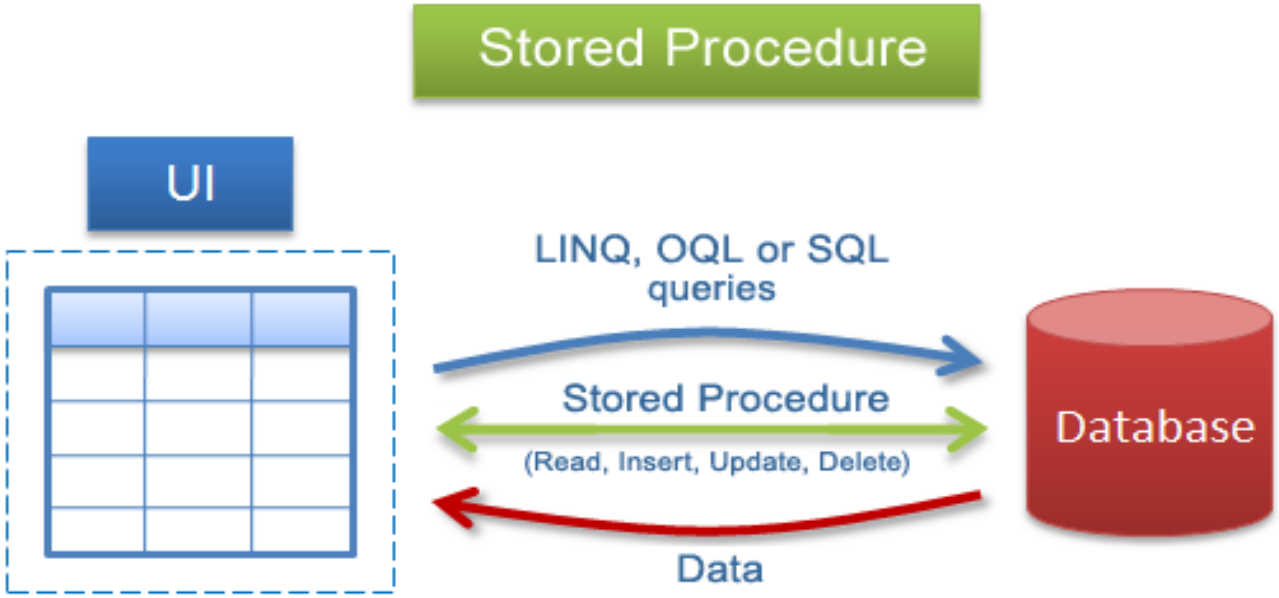
procedimiento almacenado MySQL

Los **procedimientos almacenados (*Stored Procedure*)**, son un conjuntos de **comandos o instrucciones en SQL almacenados en el servidor**, con el fin de que los clientes no tengan que ejecutar los comandos individuales. Esto nos permite trabajar con grandes cantidades de datos delimitando la complejidad a nivel de servidor y ofreciendo a los usuarios directamente lo que están buscando.

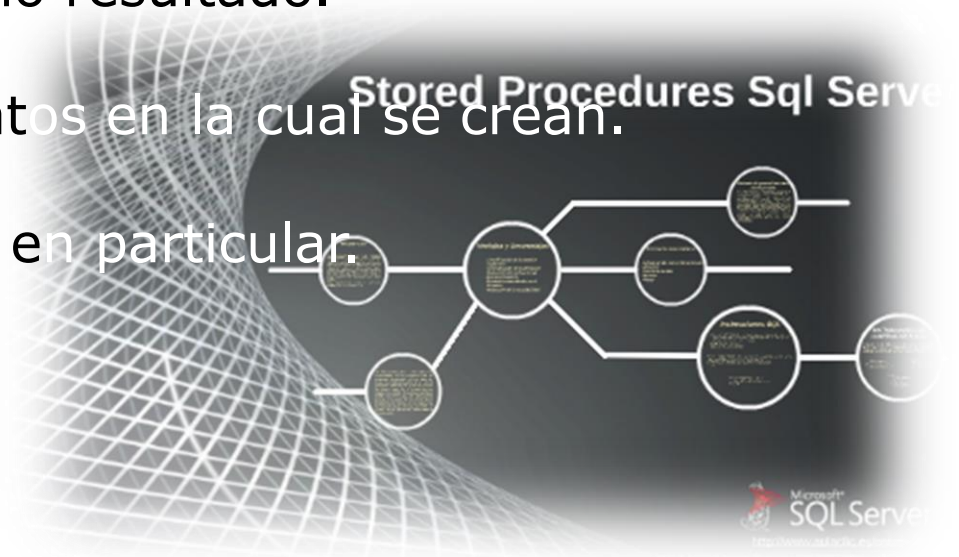
Tomado de: https://blog.mdcloud.es/procedimientos-almacenados-mysql-que-son-como-crearlos-y-ventajas/



procedimiento almacenado MySQL

Estas son las **características principales** de los procedimientos almacenados MySQL:

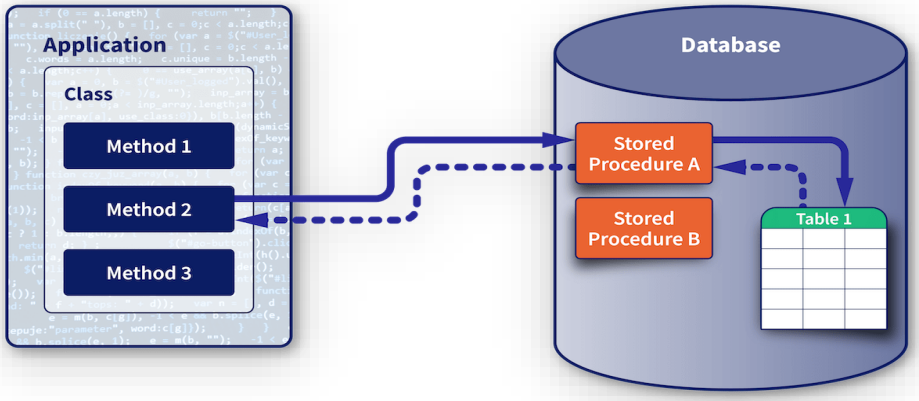
- Pueden recibir y devolver parámetros.
- Pueden manejar tablas, ejecutando operaciones e iteraciones de lectura/escritura.
- Pueden devolver una tabla como resultado.
- Se almacenan en la base de datos en la cual se crean.
- No dependen de ninguna tabla en particular.
- Pueden aceptar recursividad.



procedimiento almacenado MySQL

Procedimientos almacenados MySQL: ventajas

- **Compatibilidad**: Pueden ser ejecutados en el caso de que las aplicaciones del lado del cliente sean implementadas en diferentes plataformas en distintos lenguajes.
- **Seguridad**: Se evita el acceso directo a la BD. Los procedimientos almacenados MySQL ofrecen un entorno seguro, otorgando permisos y privilegios para la ejecución.
- **Rendimiento**: con los procedimientos almacenados MySQL todo el trabajo se ejecuta en el servidor



procedimiento almacenado MySQL

Crear procedimientos almacenados MySQL

- **CREATE PROCEDURE**: nos permite crear procedimientos almacenados MySQL para su posterior ejecución.
- **ALTER PROCEDURE**: con esta sentencia podemos modificar los procedimientos almacenados MySQL.
- **DROP PROCEDURE**: nos permite eliminar los procedimientos almacenados MySQL.
- **DELIMETER**: permite crear un delimitador para las consultas SQL.

procedimiento almacenado MySQL

Para que el usuario pueda hacer uso de los procedimientos almacenados, es necesario que la cuenta de usuario tenga permisos para:

- **INSERT,**
- **DELETE,**
- **CREATE ROUTINE** y
- **ALTER ROUTINE**.

De acuerdo al manejo que hacen de los parámetros (variables de entrada y salida), existen 3 tipos de procedimientos almacenados:

- **IN:** Es el tipo de procedimiento por default, el procedimiento utiliza parámetros que pasan el argumento por valor hacia dentro del mismo.

- **OUT:** El valor del parámetro se regresa hacia afuera de este para utilizarse ya sea en una variable u otra consulta.

- **INOUT:** Los parámetros definidos funcionan en ambas direcciones.

procedimiento almacenado MySQL

Las palabras clave a tener en cuenta:

- **DELIMITER**: Permite establecer un delimitador para que el procedimiento pueda crearse y finalizarse correctamente. Como delimitador, se puede usar casi cualquier carácter, pero los más recomendados son: punto y coma (;), signo monetario doble($$) y barra doble (//)
- **CREATE PROCEDURE**: Como su nombre lo indica, sirve para crear un procedimiento
- **BEGIN**: Marca el inicio del cuerpo del procedimiento.
- **END**: Marca el final del cuerpo del procedimiento.
- **CALL**: Lo utilizaremos para llamar al procedimiento y que este haga su trabajo.

Tomada de: https://www.cablenaranja.com/como-crear-y-usar-procedimientos-almacenados-en-mysql/

procedimiento almacenado MySQL

ESTRUCTURA PRINCIPAL

```
delimiter //
create procedure nombre_procedimiento
Begin
Instrucciones en MySQL
end //
delimiter;
```

Tomada de: https://www.cablenaranja.com/como-crear-y-usar-procedimientos-almacenados-en-mysql/

procedimiento almacenado MySQL

OPERADORES DE SALIDA

OUT: El valor de este parámetros pude ser cambiado en el procedimiento, y además su valor modificado será enviado de vuelta al código o programa que invoca el procedimiento.

Sintaxis

```
delimiter //
Create procedure nombre_procedimiento (out variable_ salida tipo)
Begin
Instrucciones Mysql;
end //
```

procedimiento almacenado MySQL

OPERADORES DE SALIDA

Usar BD de jardinería

Ejemplo: Diseñar un PA que nos muestre el monto total de pagos que ha realizado el cliente Con código igual a 1).

```
Delimiter //
create procedure monto (out valor decimal(15,2))
Begin
select sum(total) into valor
from pago
where codigo_cliente = 1;
end //
```

Para ejecución

```
call monto(@valor);
```

```
call jardineria.monto(@valor);
select @valor;
```

Salida

	@valor
▶	4000.00

procedimiento almacenado MySQL

OPERADORES DE SALIDA

Usar BD de jardinería

Ejemplo: Diseñar un PA que nos muestre la cantidad de empleados que son “Representante Ventas”).

```
delimiter //
create procedure oficinas (out cantidad int )
Begin
select count(*) into cantidad
from empleado
where puesto = 'Representante Ventas';
end //
```

Para ejecución

```
call jardineria.oficinas(@cantidad);
select @cantidad as CANTIDAD;
```

```
call oficinas(@cantidad);
select @cantidad AS 'El total es';
```

Salida

	El total es
▶	19

procedimiento almacenado MySQL

OPERADORES DE ENTRADA y SALIDA

```
delimiter//
CREATE PROCEDURE nombre (IN variable tipo, OUT variable tipo)
BEGIN
Instrucciones,
END//
```

Ejemplo: Diseñar un PA que nos muestre el monto total de pagos que ha realizado el cliente
con código igual a ????).

```
delimiter //
create procedure monto_usuario (IN codicliente INT, OUT valor decimal(15,2))
Begin
select sum(total) into @valor
from pago
where codigo_cliente = codicliente;
End//
```

```
call jardineria.monto_usuario(1, @valor);
select @valor AS 'El total es:'
```

procedimiento almacenado MySQL

MODIFICANDO REGISTROS CON P.A.

```
UPDATE nombre_tabla
SET columna1 = valor1, columna2 = valor2
WHERE columna3 = valor3
```

```
UPDATE precios
SET pedido='PK'
WHERE id = 1;
```

Diseñar un PA que incremente en un 20% cada pago que a realizado el cliente con código ????

```
delimiter //
create procedure actualiza(IN variable INT)
Begin
UPDATE pago
set total = total+total*0.20
where codigo_cliente = variable;
end //
```

procedimiento almacenado MySQL

MODIFICANDO REGISTROS CON P.A.

Diseñar un PA que incremente 2.000 pesos a cada pago que sea menor o igual a 3.000 pesos.

```
delimiter //
create procedure actualizasinparametros()
Begin
update pago
set total = total+2000
where total <= 3000;
end //
```

total
4400.00
4400.00
5000.00
5000.00
2926.00
20000.00
20000.00
20000.00
20000.00
3849.00
23794.00
4390.00
2929.00
4246.00

procedimiento almacenado MySQL

CONSULTANDO, MODIFICANDO y ELIMINANDO REGISTROS CON P.A.

Diseñar un PA que diga los días transcurridos entre la **FECHA DEL PEDIDO** y la **FECHA DE ENTREGA DEL PEDIDO**

codigo_pedido	fecha_pedido	fecha_entrega	dias
1	2006-01-17	2006-01-19	2
2	2007-10-23	2007-10-26	3
3	2008-06-20	NULL	NULL
4	2009-01-20	NULL	NULL
8	2008-11-09	2008-11-14	5
9	2008-12-22	2008-12-28	6
10	2009-01-15	NULL	NULL

Secuencia de código

```
SELECT codigo_pedido, fecha_pedido, fecha_entrega, DATEDIFF(fecha_entrega, fecha_pedido)
AS dias
FROM pedido;
```

procedimiento almacenado MySQL

CONSULTANDO, MODIFICANDO y ELIMINANDO REGISTROS CON P.A.

Diseñar un PA que consulte aquellos pedidos que tienen mas de 7 días de retraso en la entrega.

codigo_pedido	fecha_pedido	fecha_entrega	Dias perdidos
16	2009-01-06	2009-01-15	9
24	2008-07-14	2008-07-25	11
28	2009-02-10	2009-02-20	10
29	2008-08-01	2008-09-01	31
30	2008-08-03	2008-08-31	28
31	2008-09-04	2008-10-04	30
32	2007-01-07	2007-01-27	20

Secuencia de código

```
SELECT codigo_pedido, fecha_pedido, fecha_entrega, DATEDIFF(fecha_entrega, fecha_pedido)
As 'Dias de Mora‘
FROM pedido
WHERE DATEDIFF(fecha_entrega, fecha_pedido) > 7;
```

procedimiento almacenado MySQL

CONSULTANDO, MODIFICANDO y ELIMINANDO REGISTROS CON P.A.

Diseñar un PA que muestre los pedidos sin entregar que tiene cada cliente.

procedimiento almacenado MySQL

CONSULTANDO, MODIFICANDO y ELIMINANDO REGISTROS CON P.A.

Diseñar un PA que muestre de manera descendente, de mayor a menor los pedidos con mayor días de moratoria.

procedimiento almacenado MySQL

CONSULTANDO, MODIFICANDO y ELIMINANDO REGISTROS CON P.A.

Diseñar un PA que muestre los pedidos de cada cliente que no han sido entregados.

procedimiento almacenado MySQL

CONSULTANDO, MODIFICANDO y ELIMINANDO REGISTROS CON P.A.

Diseñar un PA que incremente el precio de venta de cada articulo en un 16%.

procedimiento almacenado MySQL

CONSULTANDO, MODIFICANDO y ELIMINANDO REGISTROS CON P.A.

Diseñar un PA que incremente el precio de venta con un incremento del ????.

procedimiento almacenado MySQL

CONSULTANDO, MODIFICANDO y ELIMINANDO REGISTROS CON P.A.

Diseñar un PA que incremente que elimine de cada cliente los pedidos que no han sido entregados.

procedimiento almacenado MySQL

CONSULTANDO, MODIFICANDO y ELIMINANDO REGISTROS CON P.A.

Diseñar un PA que muestre de manera descendente, de mayor a menor los pedidos con mayor días de moratoria de aquellos pedidos que tuvieron mas de 7 días transcurridos desde la fecha de entrega.

Secuencia de código

```
select codigo_cliente, codigo_pedido, fecha_pedido, fecha_entrega,
DATEDIFF(fecha_entrega, fecha_pedido) AS dias_t
from pedido
where datediff(fecha_entrega, fecha_pedido) > 7
order by dias_t DESC;
```

procedimiento almacenado MySQL

CONSULTANDO, MODIFICANDO y ELIMINANDO REGISTROS CON P.A.

Diseñe un procedimiento almacenado que actualice las fechas de entrega de pedidos que no se han efectuado, **queden para el día de hoy**

Secuencia de código

```
UPDATE pedido
SET fecha_entrega = CURDATE()
where fecha_entrega is null;
```

procedimiento almacenado MySQL

CONSULTANDO, MODIFICANDO y ELIMINANDO REGISTROS CON P.A.

Diseñe un procedimiento almacenado que actualice las fechas de entrega de pedidos que no se han efectuados, **que tengan fecha de entrega de 5 días a partir de hoy**

Secuencia de código

```
UPDATE pedido
SET fecha_entrega = DATE_ADD(CURDATE(), INTERVAL 5 DAY)
where fecha_entrega is null;
```

procedimiento almacenado MySQL

CONSULTANDO, MODIFICANDO y ELIMINANDO REGISTROS CON P.A.

Borrado de registros:

```
DELETE FROM nombre_tabla
WHERE condicion;
```

Diseñe un procedimiento almacenado que elimine los pedidos que no fueron entregados en la fecha estipulada.

procedimiento almacenado MySQL

CONSULTANDO, MODIFICANDO y ELIMINANDO REGISTROS CON P.A.

Diseñe un procedimiento almacenado que elimine los pedidos que tuvieron mas de 10 días de atraso.

```
DELETE from pedido
where DATEDIFF(fecha_esperada, fecha_pedido) > 5;
```

procedimiento almacenado MySQL

CONSULTANDO, MODIFICANDO y ELIMINANDO REGISTROS CON P.A.

Diseñe un procedimiento almacenado que actualice las fechas acordadas de entrega de pedidos en 10 días a partir de la fecha acordada de aquellos pedidos que no están entregados.

Secuencia de código

```
UPDATE pedido
SET fecha_esperada = DATE_ADD(fecha_esperada, INTERVAL 10 DAY)
where fecha_entrega is null;
```

procedimiento almacenado MySQL

Escribe un procedimiento que se llame **calcular_max_min_media**, que reciba como parámetro de entrada el nombre de la gama de un producto y devuelva como salida tres parámetros. El precio máximo, el precio mínimo y la media de los productos que existen en esa gama.

Resolver el ejercicio de dos formas distintas, utilizando SET y SELECT ... INTO.

procedimiento almacenado MySQL

```
DROP PROCEDURE IF EXISTS calcular_max_min_media;
DELIMITER //
CREATE PROCEDURE calcular_max_min_media( IN gama VARCHAR(50),
OUT maximo DECIMAL(15, 2), OUT minimo DECIMAL(15, 2),
OUT media DECIMAL(15, 2) )

BEGIN SET maximo = ( SELECT MAX(precio_venta)
FROM producto WHERE producto.gama = gama);

SET minimo = ( SELECT MIN(precio_venta)
FROM producto WHERE producto.gama = gama);

SET media = ( SELECT AVG(precio_venta)
FROM producto WHERE producto.gama = gama);
END //;
```

procedimiento almacenado MySQL

```
CALL calcular_max_min_media('Herramientas', @maximo, @minimo, @media);

SELECT @maximo, @minimo, @media;
```

- ***Solución 2. Utilizando SELECT ... INTO***

```
DROP PROCEDURE IF EXISTS calcular_max_min_media;
DELIMITER //
CREATE PROCEDURE calcular_max_min_media( IN gama VARCHAR(50),
OUT maximo DECIMAL(15, 2),
OUT minimo DECIMAL(15, 2),
OUT media DECIMAL(15, 2) )

BEGIN SELECT MAX(precio_venta), MIN(precio_venta), AVG(precio_venta)
FROM producto WHERE producto.gama = gama
INTO maximo, minimo, media;
END //
```

procedimiento almacenado MySQL

Diseñar un procedimiento almacenado de consulta que ingresando el código del cliente nos traiga el nombre y apellido del contacto (unidos en una sola columna), y nombre del cliente.

```
drop procedure IF EXISTS contacto;
Delimiter //
create procedure contacto (IN codigo INT)
Begin
select concat(nombre_contacto, '  ', apellido_contacto) AS 'Nombre y
Apellidos', nombre_cliente
from cliente
where codigo_cliente = codigo;
end //
```

procedimiento almacenado MySQL

Determinar cuantos pedidos rechazados y cuantos entregados van en lo que va corrido del año.

procedimiento almacenado MySQL

Diseñe un procedimiento almacenado que diga si hay mas pedidos entregados que rechazados.

```
drop procedure IF EXISTS cantidad;
Delimiter //
create procedure cantidad (OUT msg varchar(255))
Begin
declare entregado int;
declare rechazado int;
SET entregado = (SELECT count(estado) from pedido where estado = 'Entregado’);
SET rechazado = (SELECT count(estado) from pedido where estado = 'Rechazado’);
IF entregado > rechazado then
set msg ='La cantidad de pedidos entregados es mayor’;
else
set msg ='La cantidad de pedidos rechazados es mayor’;
end if;
end //
```

procedimiento almacenado MySQL

Diseñe un procedimiento almacenado que muestre la diferencia que existe entre los pedidos entregados y los rechazados, acompañado de un mensaje que diga cual es mayor, los entregados o los rechazados.

procedimiento almacenado MySQL

Diseñe un procedimiento almacenado que muestre el numero de línea con mayor cantidad de pedidos.

procedimiento almacenado MySQL

Diseñe un procedimiento almacenado que muestre el articulo con mayor stock en inventario.

procedimiento almacenado MySQL

Diseñe un procedimiento almacenado que muestre la diferencia entre el precio de venta y el precio colocado por el proveedor, de todos los productos.

procedimiento almacenado MySQL

Diseñe un procedimiento almacenado que muestre por un proveedor en particular, los productos ofertados, la gamma y el stock que hay del producto en inventario